

INSuRE Spring 2023 Project Proposal: Malware Similarity

Sponsor: MITRE

Frank Posluszny
MITRE
Technical Director
malwarefrank@mitre.org

Brian Ricks
UTD
Faculty
bwr031000@utdallas.edu

Jerry Teng	Kushan Singh	Payton Harmon
UTD	UTD	UTD
jerry.teng@utdallas.edu	kxs200085@utdallas.edu	pnh170000@utdallas.edu

Surekha Kumari	Haoyan Li
UTD	UTD
sxk210055@utdallas.edu	hxl210024@utdallas.edu

February 3, 2023

Contents

1	Project Summary	3
1.1	Top Level Project Description	3
1.2	Keywords	3
1.3	Terminology	3
1.4	Team Member Responsibilities	3
1.5	Estimated Project Budget	4
1.6	Summary of Key Deliverables	4
2	Executive Summary	5
3	Motivation	6
4	Literature Review	7
4.1	Malware Similarity Analysis: An adversary perspective	7
4.2	Automatic Analysis of Malware Behavior using Machine Learning	7
4.3	How's It Made: Malware Edition: University of Texas at Dallas and University of Arizona Spring 2022	7
5	Specific Aims	9
6	Research Plan/Approach	10
7	Anticipated Issues and Challenges	12
8	Broader Impact	13
9	Final Progress	14
9.1	Heuristics Phase 1	14
9.2	Heuristics Phase 2	15
9.3	Heuristics Phase 3	17
9.4	Heuristics Summary	17
9.5	DetectItEasy	17
9.6	Capa	18
9.7	CapaRules	18
9.8	FLOSS (FLARE Obfuscated String Solver)	19
9.9	Hybrid Analysis	20
9.10	Data.csv	21
9.11	String Extraction Script	21
9.12	Challenges	22
9.13	Future Work	22
10	Investigator Biographies	24

1 Project Summary

1.1 Top Level Project Description

The goal of this project is to detect similarity in malware with a new methodology proposed by MITRE, which focuses on analyzing the fingerprints left behind by the Malware Development Life Cycle (MDLC). With no clear consensus on techniques for detecting malware similarity, our team will conduct research and explore various avenues that can help determine the choices and actions made by adversaries in each stage of the lifecycle.

1.2 Keywords

Malware Similarity, Malware Analysis, Malware Development Life Cycle (MDLC), Reverse Engineering, Obfuscation, Sandboxing

1.3 Terminology

Malware: A piece of software that performs malicious actions.

Reverse Engineering: The process of recovering information about an application from the earlier phases of the development lifecycle.

Malware Analysis: The process of understanding the behavior, original, and impact of a malware sample.

Static Analysis: The analysis of programs that involves examining the code without actually executing it.

Dynamic Analysis: The analysis of programs by monitoring the program when it is executed.

MDLC: A new malware analysis methodology developed at MITRE that aims to explore the human element of malware development and how it can be exploited to both gain information about malware samples and identify malware developers.

Sandboxing: A process where code is executed in a safe, isolated environment that attempts to mimic a realistic user's environment. Sandboxing is used to determine the program's behavior when executed without potentially compromising production environments.

Artifact: A tangible observation or pattern that can be extracted from a program that is indicative of choices made in the program's creation.

Operation Security (OPSEC): A process of denying adversarial progress by identifying and implementing protections.

Obfuscation: Techniques used to delay or prevent analysis of a malware sample by making it harder to read or understand.

1.4 Team Member Responsibilities

Kushan Singh will serve as the team lead for the project. He will be responsible for ensuring that the team will meet the milestone deadlines and will be contacting our technical director for project updates and guidance. He has experience in source code analysis and in reading assembly code. He is also well versed in Python and C/C++ programming which will be leveraged to build custom scripts.

Jerry Teng is the assistant team lead for the project. He is responsible for setting up times for the team to meet and collaborate internally. Jerry has experience with reverse engineering and malware analysis and will be assisting with the static and dynamic analysis of malware. He also has extensive practice in Python programming, so he will be aiming to automate as many steps as possible along the way to make the project more scalable.

Payton Harmon will serve as one of the principal researchers for the project. He has experience working with reverse engineering and malware so he will assist with methods for extracting artifacts from malware samples. He will also assist with overall research and planning of one of the approaches.

Surekha Kumari will be responsible for research based on Operating Systems and Networks. She has experience coding in Java and python as well as scripting in shell. She is proficient with data structures and will focus on incorporating her knowledge into malware analysis.

Haoyan Li will serve as the researcher for the project. She will be collecting data and information for malware analysis research. She will also evaluate and measure the assistant and application of machine learning in malware similarity with her knowledge in the machine learning area. Also, she will assist in the similarity matrix analysis between artifacts within the MDLC model.

1.5 Estimated Project Budget

In terms of time budget, this project is estimated to consume around 533 hours, accounting for 8 hours per week per researcher and 1 hour per week of the technical director's time. Currently, there are no monetary expenses required for the project, as we are focusing on conducting more research to better understand the various avenues of our proposed statements. We do not think that there will be any software in the implementation phase that will cost money.

1.6 Summary of Key Deliverables

Key Deliverables	Proposed Due Date
Project Proposal	2/3
Literature Review	2/10
Linux and Windows VM setup	2/24
Phase One - Complete research and decide which approaches to implement	3/3
Midterm Presentation	3/24
Midterm Report	3/26
Phase Two - Implementation of approaches	4/10
Final Presentation	5/5
Final Report	5/7

2 Executive Summary

The Malware Development Life Cycle (MDLC) represents the many actions and choices made in creating, operating, collecting, and analyzing the malware. It is divided into 8 stages - Requirements, Design, Code, Build, Configure, Package, Collection and Analysis - with the first 6 stages controlled by an adversary and the last two stages performed by defenders. Each stages of the lifecylce leave fingerprints on the malware and can be useful in determining similarity. There can be simmlarity in code, behavior, authorship and operation.

Similarity in code can include code reuse, shared libraries, api calls, algorithms and data structures. Similarity in behavior reperesents the type of malware such as keyloggers, ransomware, trojans etc. Similarity in authorship represents malware written by the same attacker or threat group. Similarity in operation represents the usage of tools in the same unique way. Our efforts will be placed into gaining a better perspective of the humanistic elements of malware from conception to analysis. To this end, we will examine various research paths to extract similarity features and map them to the MDLC.

3 Motivation

Malware in the modern era has to strongly consider the methods in which it conceals itself from the many pieces of technology that attempt to halt its execution. Malware authors use methods of concealment which include encrypting the payload, polymorphism and metamorphism (changing or re-writing the code to appear as a different signature). One of the more elaborate techniques of concealing itself is packing, where the payload is placed in a special part of the executable, and then unpacked by a small piece of code that is placed in the normal section of the executable. This poses a challenge as it can be difficult for analysts to know when they are dealing with similar malware samples. Thus, it is of great significance to detect, trace and prevent malware attacks by extracting structural information from samples for similarity analysis and mining the homologous relationship in samples, especially with the development of advanced persistent threat (APT) attack technology.

4 Literature Review

4.1 Malware Similarity Analysis: An adversary perspective

The article talks about Malware Development Life Cycle (MDLC) as a new methodology to describe the tracks left by malware creators and operators. It is based on the traditional Software Development Life Cycle and consists of 8 phases: Requirements, Design, Code, Build, Configure, Package, Collect, and Analyze. It further describes each of the stages and provides examples of choices made during each stage that may be observed through malware analysis. MDLC language can be integrated into existing frameworks to understand the adversary's capabilities and link intrusion events.

From an malware analysis practitioner's perspective, malware similarity can be identified with patterns in coding style, authorship, behaviour of the malware or same operational pattern. By revealing the connections between different malicious programs, practitioners can better understand the adversary's goals and tactics. This data with the help of MDLC and Diamond Model of Intrusion can assist development of better defenses and mitigation strategies, making it more difficult for the adversary to achieve their objectives.

In conclusion, Malware Development Life Cycle (MDLC) is an approach to understanding malware similarity. The MDLC first six stages are controlled by adversaries, including Requirements, Design, Code, Build, Configure, and Package. The two stages not controlled by the adversary are Collection and Analysis, which also have an impact on the malware. The MDLC encompasses and expands on current views of malware similarity and can be used to enhance existing frameworks. Further research is needed to expand the concepts introduced in the paper and determine the measurable artifacts associated with each stage of the MDLC.

4.2 Automatic Analysis of Malware Behavior using Machine Learning

In this article, the author provides a tool, which is capable of processing the behavior of thousands of malware binaries on daily basis, by automatically identifying novel classes of malware with similar behavior and assigning unknown malware to these discovered classes.

The framework contributes in the scalable clustering and classification, the incremental analysis of malware behavior and the extensive evaluation with real malware. Although it does not generally eliminate threats of malware, it provides an instrument for timely development of antimalware products, capable of automatically and efficiently characterizing novel breeds of malware development on a regular basis.

The frame work is first executed and monitored in a sandbox environment, where a sequential report is generated for each binary. Then the sequential reports are embedded in a high-dimensional vector space in order to geometrically asses the similarity of behavior. Then for identifying novel and known classes of malware, machine learning techniques for clustering and classification are applied. Finally the embedded behavior of malware is analyzed incrementally by alternating between clustering and classification steps.

4.3 How's It Made: Malware Edition: University of Texas at Dallas and University of Arizona Spring 2022

The authors have introduced the Malware Development Life Cycle (MDLC) as a new approach to understanding malware similarity. The MDLC consists of eight stages, with six of them being controlled by adversaries, including Requirements, Design, Code, Build, Configure, and Package. The two stages not controlled by the adversary are Collection and Analysis, which also have an impact on the malware.

The motivation for this study was to analyze the deluge of 450000 new malware samples per day and the attacker's intentions in creating these samples. To find a malware analysis solution, the team

toiled on characterizing malware engineering decisions and generating ground truth samples.

Post all the limitations, team implemented four ground truth samples, two in Java and two in C++ with six hundred and two thousand lines for final sample respectively. The implementation required extensive research in various APIs, dozens of decisions, software engineering expertise, and debugging efforts. The team concluded python as the programming language for the tool. They decided on different Python-based tools for varying identified malware compiler information. They shared their research on existing Toolchains by Detected Language. Each tool in the toolchain would require a specialized file or dataset to match information artifacts to the MDLC. The suggested flow for each tool in the article is receive and parse output, then process as per MDLC. Unpack if necessary and reanalyse.

The paper also mentioned the effort as colossal and impossible for a four-person team to accomplish the entirety of potential design choices. They agreed further research is needed to expand the concepts introduced in the paper and determine the measurable artifacts associated with each stage of the MDLC.

5 Specific Aims

The high level goal of this project is extracting artifacts from malware that can be mapped to various phases of the MDLC. To reach this end, we have conjectured four research paths that we plan on exploring before the midterm, upon which we will decide which ones are the most feasible and implement them. A combination of these attributes from each path can be combined to provide a better characterization of similarity features.

The first approach involves on researching the later stages of the MDLC (i.e. Build, Configure, Package) and using artifacts extracted from those stages to develop similarity heuristics. Since much research and development has already been done on extracting this type of data from binaries, this path will focus on analyzing artifact combinations and/or sequences of artifacts that may indicate a behavior pattern. More unique indicators from artifacts will be explored, such as specific command line arguments or packing methodologies. The tools used to extract artifacts can be existing industry tools, as well as previous work done by the Spring 2020 UTD group.

The second approach places more emphasis on the earlier phases of the MDLC, (i.e. Requirements, Design). In this path, we assume that we begin with a sample that is successfully unpacked and possibly decompiled, and thus we have the bytecode/source code. In theory, the code would contain traces of the malware's original requirements and the design choices that went into it. The aim is to understand the human elements that influenced the development of the code. We will also be looking for traces of OPSEC in the code phase and mapping it to requirements or design choices.

The third approach is to look into existing research on automated similarity analysis and see how it can be incorporated with the MDLC. This part is largely focused on dynamic analysis and finding a way to utilize the output of various sandboxes. An example is the Malheur project [3], which uses vectorized system calls to train clustering and classification models to categorize and predict malware behavior types. Another source of information we could look at is online/offline sandboxing tools. These sandboxes gather information from both static and dynamic analysis and are fairly accessible. More research would have to be done on other possibilities and the specifics of the integration, but this could provide a more scalable solution for malware similarity.

The final approach is finding patterns in obfuscation techniques that can be indicative of certain authors or groups. For example a malware author could apply multiple layers of obfuscation that is not commonly used by other writers. Cases like these could either be used as a direct or supplementary indicator to help categorize cases.

We think that all of these approaches have merits and the end result may contain results from multiple approaches.

6 Research Plan/Approach

Our project will consist of a three phases. The first phase will begin after the project proposal is complete and consist of each member conducting a literature review of the possible approaches to the problem described above in section 5. During this phase we will work with each other and our mentor to report any findings and pitfalls with the possible solutions. These findings will be saved and discussed at our weekly meetings. We hope to come out of the first phase with a greater understanding of the problem space and a determination on which of the solutions we would like to pursue implementation of.

The second phase will consist of prototyping the approach we decide on. Given that we will not know the most effective solution until after phase one, it is impossible to determine specifics of what the stage will look like; however, the approach will likely consist of a design period where the team determines the steps taken in the solution, and then an implementation period where we enact the design plan. The approach used in phase two will likely be one of the approaches described in section 5 or a new approach found during the literature review. At the end of this phase, we hope to have a working prototype or case study of our approach to present our findings on.

In the third and final phase we will be conduct a review of our initial approach and revise it as necessary. This will be one of the shortest phases as we expect most of the semester will be spent conducting research and implementing a solution. During this phase we hope to address any issues or bugs associated with our chosen method of solving the problem. We also hope to identify any potential pitfalls with our approach that can be addressed in future research in detecting malware similarity. The planned timeline of each phase can be found in section 1.6 and a more detailed view of the research timeline can be seen in the figure below.

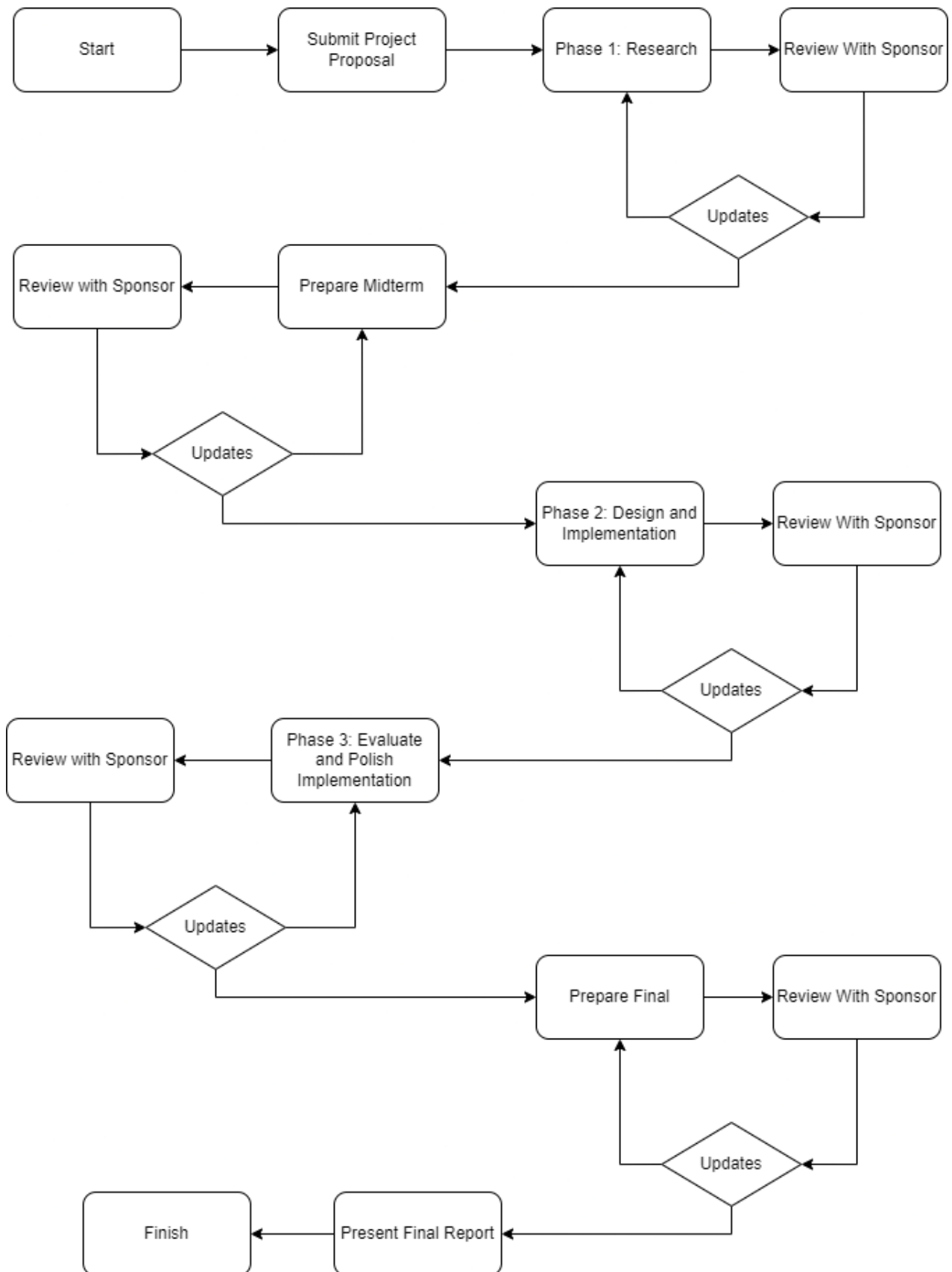


Figure 1: Research Plan Flowchart

7 Anticipated Issues and Challenges

This project will likely face many challenges making it important for us to anticipate and plan for as many as possible. One of the majors problems we anticipate is the overall inexperience of the team in malware analysis and reverse engineering. The team comes from a varying degree of technical backgrounds which means that there will be a steep learning curve for each of us as we are faced with new topics that we may not have been exposed to yet. Besides the learning curve, we anticipate that there is a possibility that none of our proposed approaches have a strong basis for extracting artifacts in the MDLC and utilizing it for malware similarity. If this occurs, we hope that it will occur in the first phase where we can use our research to pivot into a new and more effective approach.

8 Broader Impact

Malware analysis is the analysis of a suspicious file by using tools and procedures in order to understand its behavior and purpose. The team is focusing on similarity analysis between different malware, which could be helpful for security teams rapidly detect potential malicious objects before any destruction happens within the organization. By understanding similarities between malware, ultimately, the team's research will exert impact on improving the accuracy of categorizing and evaluating malware attack incidents.

Moreover, malware similarity analysis also benefits malware analysis automation. Over the years, malware evolves not only in complexity but also diversity and quantity. A better understanding of malware similarities can improve the speed and accuracy of malware detection automation.

9 Final Progress

After a few weeks of research and further literature review, we proposed a few ideas about what kind of artifacts we would want to look at to our sponsor. The approach we finalized was analyzing usage strings and command line arguments from malware to eventually determine a similarity metric or make an educated guess about the functionality of the malware. To help with this, our sponsor provided us with a set of malware samples that were known to contain command line arguments from previous analysis. Our goal is to analyze these samples in depth to determine a way to extract the usage strings as well as create a heuristic that can be used to find other samples that accept command line arguments.

Malware Name	Functionality	Heuristic Found/Rule exists
AdFind	Query Active Directory	No
NirSoft	Extract passwords saved in browser	Yes
Onhat	Network proxy	Yes
Ransomware	Assorted Ransoms	No
RemCom	Remote Administration Tool	Yes
Upchurch Group 2	Network proxy	Yes
X-tunnel	Network proxy	No

Table 1: Malware Samples

Our first step after receiving the samples was to setup the project infrastructure. This involved installing and configuring two virtual machines (VM) namely FlareVM and Remnux for the purposes of malware analysis. FlareVm is an open source Windows based security distribution while Remnux is available for Linux. Each of the virtual machines were setup such as that they were isolated from the public network and no communication was allowed between the VM and the host.

We analyzed bytecode of tools that accept command line-arguments looking for patterns that indicate a program accepts command-line arguments and developed a script to perform large-scale information gathering for portable executable files. This custom script provides output in a CSV format, which can be ingested by machine learning models or other automated tools.

9.1 Heuristics Phase 1

This phase involved analysis of pre-main initialization actions with focus on the following - finding the Windows API function that invokes command-line parsing and loads command-line string into memory: `_getmainargs`, `_wincmdln`, `GetCommandLineA/W`, etc, tracking the location of the command-line string until main is called, and noting the way that `argv` is passed to main (stack or register based). We could not assume standard calling conventions due to compiler differences, intentional obfuscation or manual assembly programming.



Figure 2: Unconvention Calling Convention

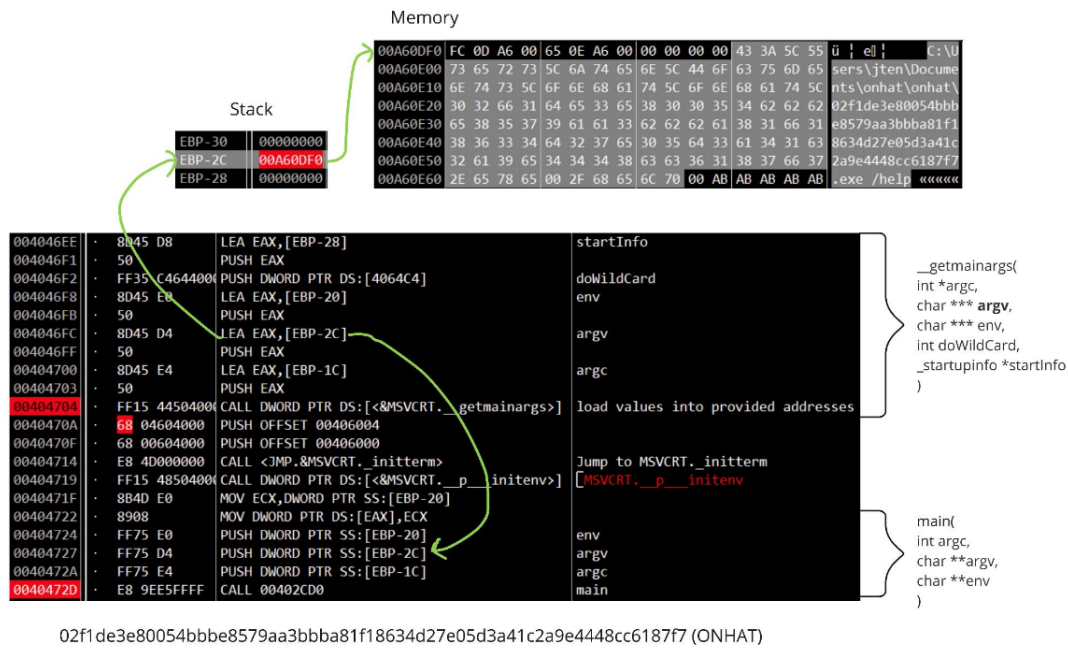


Figure 3: Tracking the origin and initialization of argv before main

9.2 Heuristics Phase 2

As the main function performs its function prologue, we needed to maintain knowledge of the location of argv. For stack-based calling conventions, this involved emulating the stack allocation operations so we could tell when argv is read relative to ESP or EBP. Traditionally, function arguments are referenced in assembly using an offset to EBP. However, our justification for this phase is that

we've noticed a different way that functions get the address of `argv`: by referencing with respect to `ESP`, as shown in Figure 5. In these cases, if we didn't keep track of `ESP` during the function prologue, we wouldn't know that `0x2B0` was subtracted from `ESP` and that `argv` is being accessed.

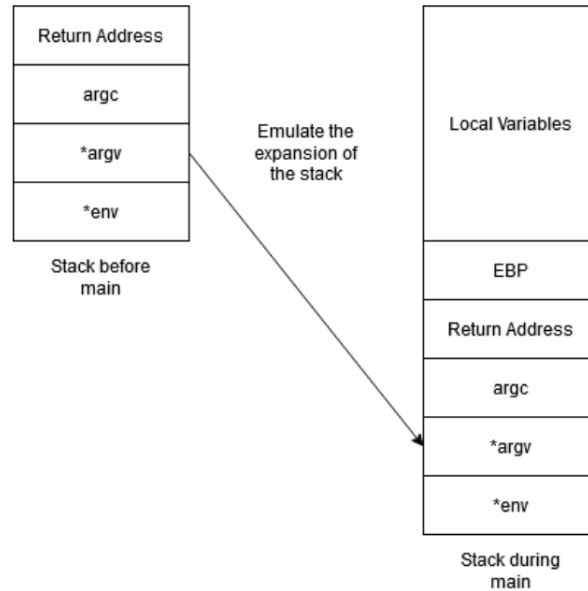
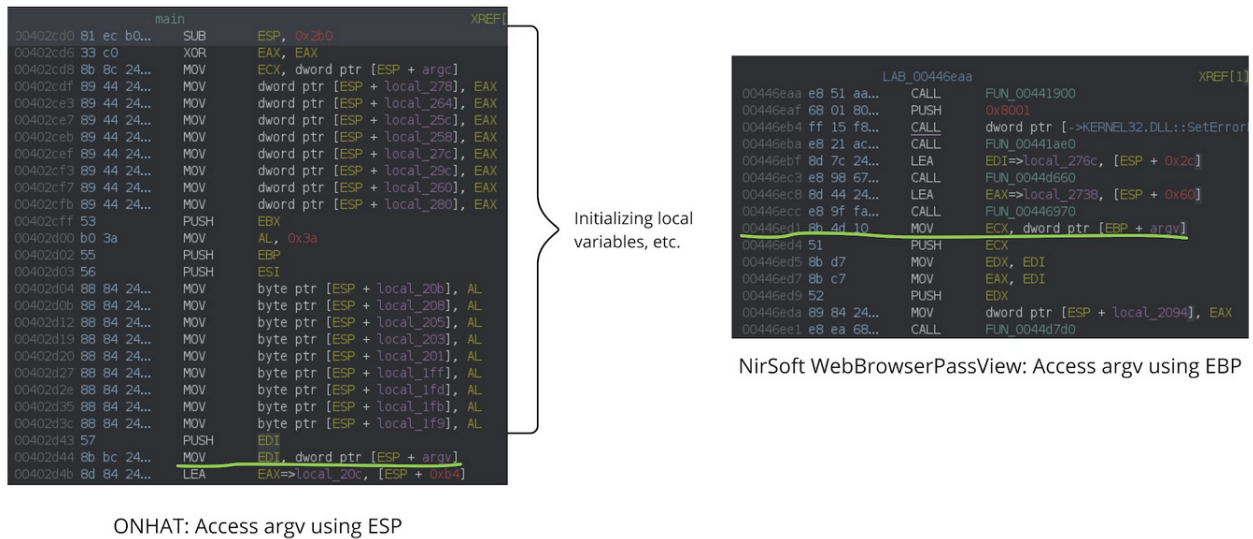


Figure 4: Emulating the function prologue

Figure 5: `argv` accessed relative to `ESP` vs `EBP`

9.3 Heuristics Phase 3

The final phase involved detecting any read references to the location of argv that may indicate it is being used. For example, in the Figure 6, we can see a common theme of the address of argv being loaded from its location on the stack into a register. This register then gets pushed onto the stack as the argument for a function that does some sort of parsing or processing with argv. For a register based calling convention, this step could involve moving the address of argv from a register into another register that is intended to be the callee's argument. This is the phase that could use the most generalization and future research, so we can capture more ways that argv is read or modified, such as argv processing directly in main instead of in another function.



Figure 6: Heuristics Phase 3

9.4 Heuristics Summary

Even though it can be hard to define exact patterns due to compiler differences, intentional obfuscation, system differences, library differences, etc, we created a general heuristic for how to track the usage of argv which can be used in conjunction with other heuristics to gain higher coverage. Capa already detects GetCommandLineA usages within main. Writing a rule for our heuristics would require a tool with advanced program tracing/emulation techniques.

In the next section we discuss the tools that we leveraged during our project and dive deeper into their functionality.

9.5 DetectItEasy

It is a free and open-source tool that can be used by software developers, security researchers, and malware analysts to determine the origin and characteristics of a binary file. DetectItEasy uses a combination of techniques to analyze binary files, including signature scanning, static analysis, and heuristics. It can detect a wide range of programming languages, packers, and compilers, including those that are commonly used to create malware. It also provides detailed reports that include information such as the file size, entropy, and section names.

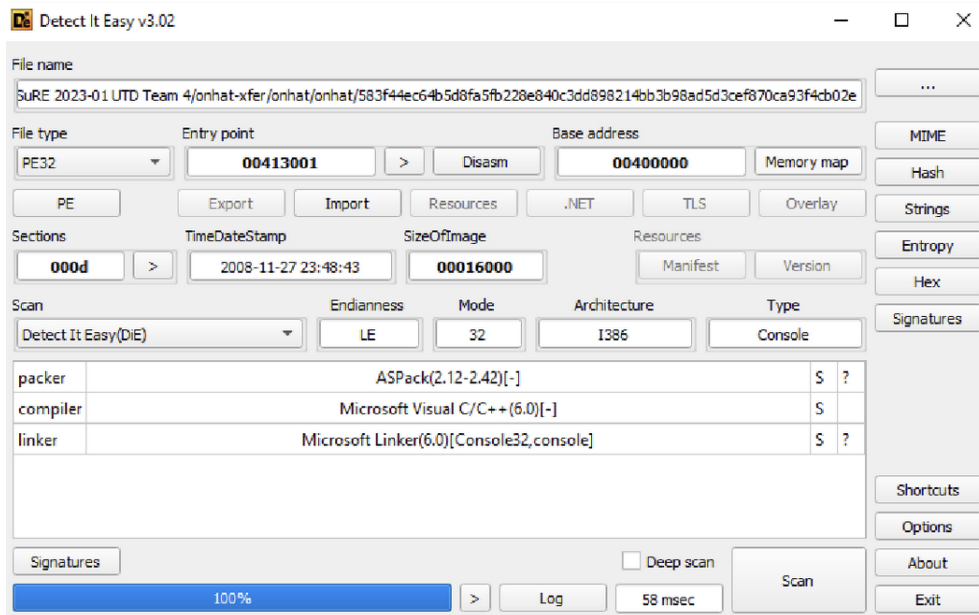


Figure 7: DetectItEasy

9.6 Capa

It is an open-source tool used for analyzing and identifying patterns in malware. It is designed to help security researchers and analysts understand the behavior and capabilities of a given malware sample. Capa works by analyzing the characteristics of a malware sample and generating a set of rules that describe its behavior. These rules can be used to identify similar malware samples or to detect instances of the same malware family. The tool uses a combination of static and dynamic analysis techniques to identify the capabilities of the malware. It can detect a range of behaviors, including file manipulation, network communication, process creation, and more. Capa is written in Python and can be run from the command line. It supports a range of file formats, including Windows executables, DLLs, and scripts.

9.7 CapaRules

This slide shows us how to create CapaRules which are YAML documents that contain metadata and a tree of statements to express their logic. A capa rule uses a structured combination of features to describe a capability that may be implemented in a program. If all required features are present, capa concludes that the program contains the capability. The rule language supports logical operators and contains. In the example, The “create TCP socket” rule says that the numbers 6, 1, and 2, and calls to either of the API functions socket or WSASocket must be present in the scope of a single basic block. Basic blocks group assembly code at a very low level making them an ideal place to match tightly related code segments. Capa also supports matching at the function and the file level within basic blocks. The function scope ties together all features in a disassembled function, while the file scope contains all features across the entire file.

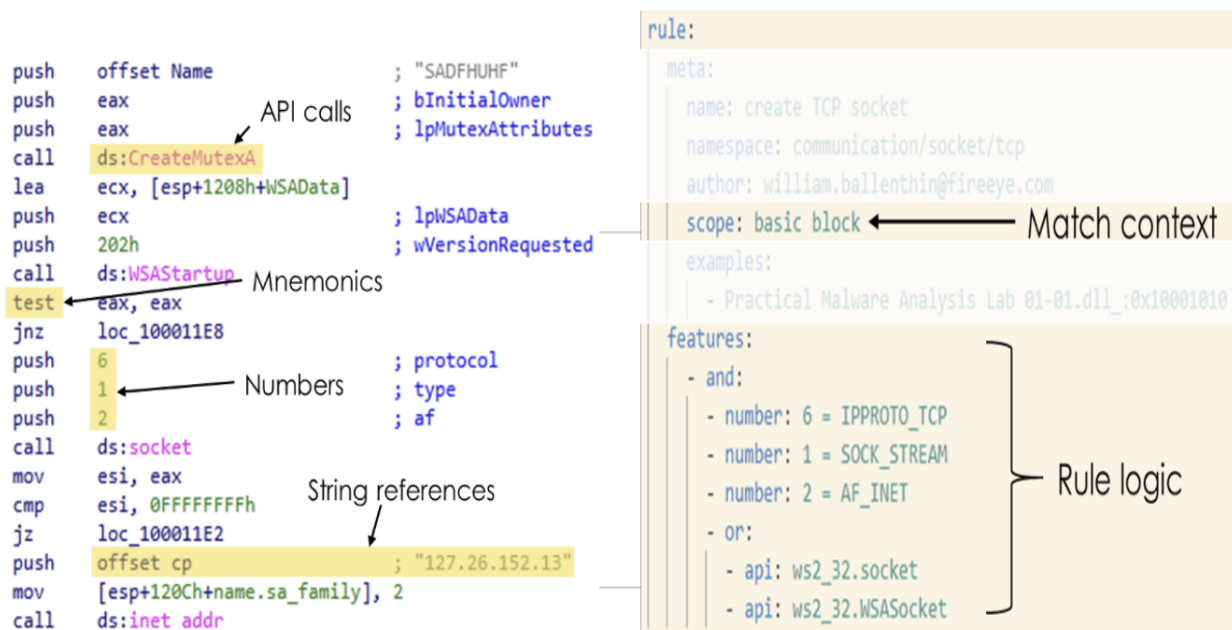


Figure 8: CapaRules

9.8 FLOSS (FLARE Obfuscated String Solver)

Mandiant FLOSS is a tool used for analyzing and extracting information from malware. It is designed to help security researchers and analysts understand the behavior of a given malware sample and identify indicators of compromise (IOCs) that can be used to detect and mitigate future attacks. FLOSS works by extracting and analyzing the strings, functions, and other data within a malware sample. It can identify code snippets and libraries that have been used to create the malware and generate a report that describes its behavior. One of the key features of FLOSS is its ability to handle a wide range of file formats, including Windows executables, DLLs, scripts, and more. It can also detect obfuscated and encrypted strings and extract the relevant information. FLOSS is available as a command-line tool and can be run on Windows, Linux, and macOS. It supports a variety of output formats, including text, JSON, and STIX.

```

FLARE FLOSS RESULTS (version v2.2.0-g783dd8f)
-----
| file path      | C:\Users\Flare\Desktop\Malware Samples\INGuRE 2023-01 UTD Team 4\onhat-xfer\onhat\onhat\02Fid
| extracted strings |
| static strings  | 462
| stack strings  | 17
| tight strings   | 0
| decoded strings | 3
-----

| FLOSS STACK STRINGS (17) |
CONNECTS SERVER ERROR[%d].
[ONHAT] CLOSSES EVENT (%d, %d).
NoHAT
nohat
nohat
NoHAT
b:ef:h1:o:O:p:P:s:t:
ONTAH_ENV_THRU_BIND
b:ef:h1:o:O:p:P:s:t:
ONTAH_ENV_ONTAH
ONTAH_ENV_SERV_PORT_X
ONTAH_ENV_THRU_PORT_X
ONTAH_ENV_SERV_BIND
ONTAH_ENV_SERV_PORT
ONTAH_ENV_THRU_PORT
ONTAH_ENV_SERV_ADDR
ONTAH_ENV_THRU_ADDR

| FLOSS TIGHT STRINGS (0) |

| FLOSS DECODED STRINGS (3) |
nohat
NoHAT
NoHA

```

Figure 9: FLOSS

9.9 Hybrid Analysis

Hybrid Analysis is a security tool used for automated analysis of malware samples. It combines both automated and manual analysis techniques to provide comprehensive information about the behavior and impact of the malware on a system. It executes the malware within a sandbox environment, to capture the system-level behavior of the malware such as file creation, registry modification, network traffic, and system calls. We found that Hybrid Analysis was able to extract strings on the stack which the other tools were not able to.

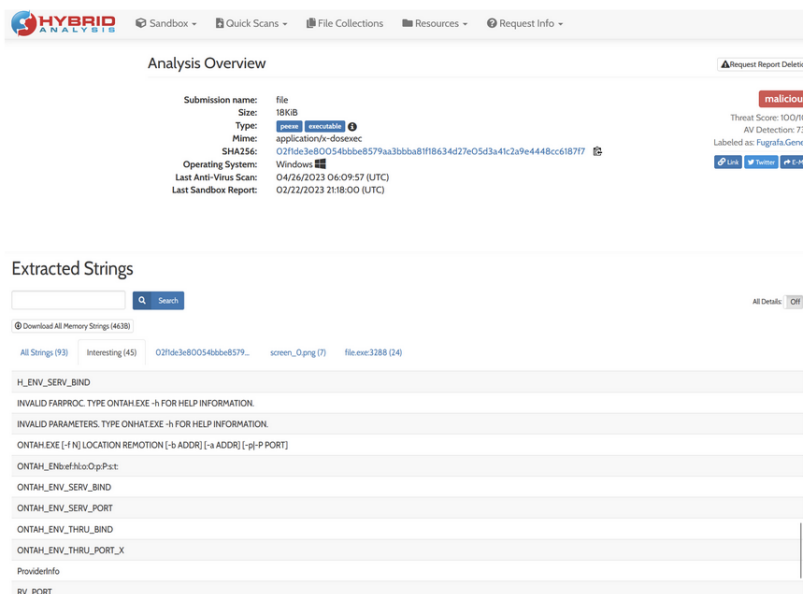


Figure 10: Hybrid Analysis

9.10 Data.csv

We developed a bash script to automate the string extraction from malware by invoking the previously highlighted tools. The output of the script is a csv file which can be used for further analysis and automation. The first column is the hash of the malware sample, the second column signifies whether the malware is packed or not, and the remaining columns contain the output of the tools we have used.

```

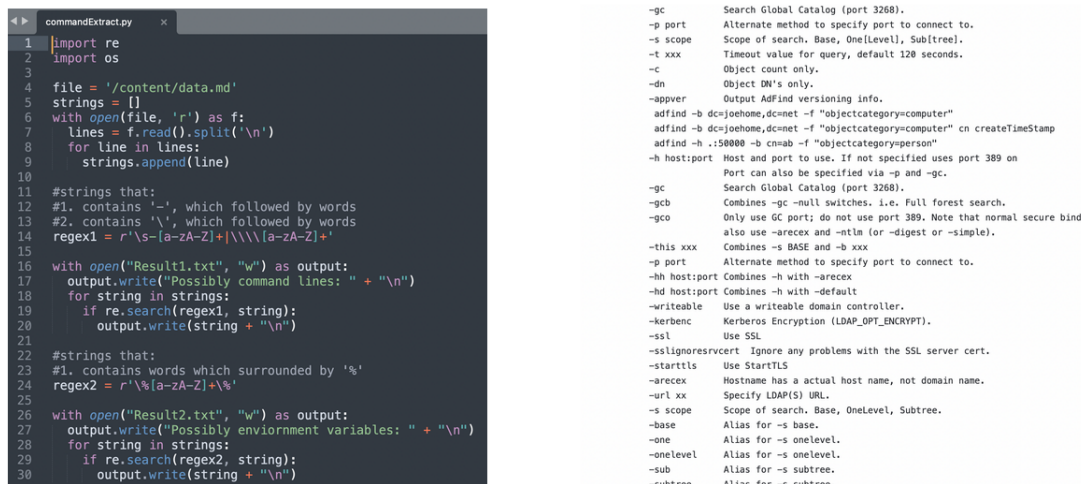
data.csv
1 hash,is_packed,pefeat,floss,capa
2 23873bf2670cf64c2440058130548d4e4da412dd,False,"Filename: 23873bf2670cf64c2440058130548d4e4da412dd
3 Program language not recognized. This program currently checks for Java, Python, and Go executables.
4 Programming Language: Unknown
5 Timestamp: 2012-08-09 14:56:49
6 File Type: PE32 executable (console) Intel 80386, for MS Windows
7 Machine: 64-bit (x86-x64)
8 Magic: 32-bit binary
9 Metadata:
10 Crypto:
11 Antidebug:
12   GetLastError
13   IsDebuggerPresent
14   TerminateProcess
15   UnhandledExceptionFilter
16 Antivm:
17 Packers:
18   Obsidium_v10061
19   VC8_Microsoft_Corporation
20   Microsoft_Visual_Cpp_8
21   Visual_Cpp_2008_Release_Microsoft
22 Files:
23   USER32.DLL                               Library
24   KERNEL32.dll                             Library
25   ADVAPI32.dll                             Library
26 URLs:
27 IPs:

```

Figure 11: Data.csv

9.11 String Extraction Script

The string extraction script is a custom script built to parse command line strings. Regular expressions is implemented in extracting and parsing possible command lines from given strings. It can be seen that we successfully extracted some useful command lines and environment variables by matching string patterns. The code is simple and easy to implement. However, it does not pre-process or filter the strings and the extracted command lines may contain some unnecessary content. It is expected to implement machine learning to get a better result if a large and correct data set can be provided to achieve this functionality.



```

1 import re
2 import os
3
4 file = '/content/data.md'
5 strings = []
6 with open(file, 'r') as f:
7     lines = f.read().split('\n')
8     for line in lines:
9         strings.append(line)
10
11 #strings that:
12 #1. contains '-', which followed by words
13 #2. contains '\', which followed by words
14 regex1 = r'\S-[a-zA-Z]+\|\\\\[a-zA-Z]+'
15
16 with open("Result1.txt", "w") as output:
17     output.write("Possibly command lines: " + "\n")
18     for string in strings:
19         if re.search(regex1, string):
20             output.write(string + "\n")
21
22 #strings that:
23 #1. contains words which surrounded by '%'
24 regex2 = r'%[a-zA-Z]+%'
25
26 with open("Result2.txt", "w") as output:
27     output.write("Possibly environment variables: " + "\n")
28     for string in strings:
29         if re.search(regex2, string):
30             output.write(string + "\n")

```

```

-gc Search Global Catalog (port 3268).
-p port Alternate method to specify port to connect to.
-s scope Scope of search. Base, OneLevel, Subtree.
-t xxx Timeout value for query, default 120 seconds.
-c Object count only.
-dn Object DN's only.
-appver Output Adfind versioning info.
adfind -b dc=joehome,dc=net -f "objectcategory=computer"
adfind -b dc=joehome,dc=net -f "objectcategory=person" cn createTimeStamp
adfind -h .:50000 -b cnab -f "objectcategory=person"
-h host:port Host and port to use. If not specified uses port 389 on
Port can also be specified via -p and -gc.
-gcb Search Global Catalog (port 3268).
-gcb Combines -gc -null switches. i.e. Full forest search.
-gco Only use GC port; do not use port 389. Note that normal secure bind
also use -arecx and -ntlm (or -digest or -simple).
-this xxx Combines -s BASE and -b xxx
-p port Alternate method to specify port to connect to.
-hh host:port Combines -h with -arecx
-hd host:port Combines -h with -default
-writable Use a writable domain controller.
-kerbenc Kerberos Encryption (LDAP_OPT_ENCRYPT).
-ssl Use SSL.
-sslignorecert Ignore any problems with the SSL server cert.
-starttls Use StartTLS
-arecx Hostname has a actual host name, not domain name.
-url xx Specify LDAP(S) URL.
-s scope Scope of search. Base, OneLevel, Subtree.
-base Alias for -s base.
-one Alias for -s onelevel.
-onelevel Alias for -s onelevel.
-sub Alias for -s subtree.
-subtree Alias for -s subtree.

```

Figure 12: String Extraction Script

9.12 Challenges

Throughout this project we have encountered many challenges. Early on in the project we were met with difficulty setting up our environment. In order to analyze malware, it is best practice to set up isolated virtual machine with the analysis tools already on them. This, however, is not possible for some of our teammates who use M1 Macs as their daily driver. We looked into other alternatives on M1 macs but the technology for emulating x86 machines just isn't ready yet. In the end we got around this by delegating tasks requiring malware analysis to those who had access to the tool necessary.

Another issue we ran into was the lack of domain knowledge. Malware research is a wide field and only a few members had any experience with it. Thanks to the help of our director we were able to manage, but it certainly made it harder to progress quickly on our project. Once we had our tools setup, we had to find malware samples to analyze. With our limited knowledge on malware collections we were able to utilize the help of frank again to get a collection of 10 or so samples as a basepoint to do our analyze on.

Last, once we had our samples we ran into many challenges extracting and applying heuristics. Most of the heuristics we were looking for applied at the code level which is challenging to properly capture. Say we have a piece of code that references argv from the command line input. Depending on the compiler, the way that argv is setup can be wildly different meaning that even through we can see a heuristic on one program. The way the same heuristic is compiled in another program may be completely different.

Furthermore, if we are able to extract a list of heuristics, many open source tools such as CAPA rely on signature based detection and don't take into account potential false flags or uncertainty that come with individual traits.

9.13 Future Work

All the challenges we discussed leave a lot of room for future work and many projects. There are two branches that future teams can approach when extending our work. First they can improve on the heuristics we've found. This might look like further static and dynamic analysis of identification known samples to help identify more heuristics. It could also be further improvement on the automated script we wrote to extract strings and use it to identify help messages or any other usage strings. And last, one approach that might help is using symbolic execution to help identify structures in the code that could be used as a heuristic for command-line input.

Another branch future researchers can use is finding better ways to identify samples with command-line input automatically. This could be taking known heuristics and finding a way write CAPA/Yara rules with them. However, many of our heuristics don't fit nicely into signature based detection tools like capa or yara. It might be more useful to create a machine learning model that can take in a collection of heuristics and provide a probability that a sample accepts command-line input.

10 Investigator Biographies

Jerry Teng is in his last semester of his MS in Computer Science at The University of Texas at Dallas, where he also received his BS in Computer Science. He has past work experiences in the industry, with internships in Software Development at RealPage, Vulnerability Management at CrowdStrike, Cloud Engineering at Zscaler, and Security Engineering at Flatiron Health. Through these experiences, he has learned much about developing security automation tools to ensure secure network and cloud configurations. He has knowledge in setting up automated deployment of secure AWS cloud infrastructure using Terraform, static code analysis tools, and CI/CD technologies, such as Jenkins, GitHub Actions, and GitLab CI. He also previously worked as a researcher in UTD's System Security Lab, where he led a research project focusing on solving the problem of Python bytecode decompilation using natural language processing models. Within security, Jerry has also had past coursework and hobbyist experience in pentesting, web hacking, reverse engineering, and binary exploitation. The languages and technologies he enjoys to use for his work include Python, Golang, and Terraform.

Kushan Singh is a second-year Computer Science graduate student at The University of Texas at Dallas with a Bachelor's in Information Science and Engineering from India. He has worked as a Backend Developer managing and creating user authentication and access controls, maintaining the database and storing secure passwords, and building a REST API. In his most recent internship as an Application Security Analyst he has gained experience in penetration testing web application, code reviews, static and dynamic code analysis, and secure code practices. He has taken various security courses as part of his graduate degree such as Network Security, System Security and Malware Analysis, and Data and Application Security. He is well versed in building web applications with the MERN stack, and has development experience with C/C++ and Python as well. He is familiar with the Intel x86 architecture set and has worked with tools such as GDB, Ghidra, Burp Suite and Metasploit. He has also participated in various Capture the Flag (CTF) events.

Payton Harmon is a first-year Computer Science graduate student with a Bachelors in Software Engineering from the University of Texas at Dallas. His background is working as a Software Developer and Cybersecurity Researcher for both private industry and the US Government. His most recent position was as an intern with Sandia National Laboratory, where he reversed engineered portions of the Windows kernel and researched methods of improving cyber detection and visibility under the MITRE ATT&CK Framework. In addition to his work, Payton is the current president of his university's cyber security club where he has given talks on securing operational technology and reverse engineering. He has also competed in and facilitated multiple CTFs. Payton has taken courses in network security, cloud computing, systems security and malicious code analysis, and multiple courses in C/C++ development. In addition, Payton has experience programming in Python, and working in Amazon's Web Services.

Surekha Kumari is a second-year Computer Engineering student at the University of Texas at Dallas. She has a Bachelor's in Computer Science from India. She worked at Accenture for four years with a Telecommunications client and has experience developing Web APIs, rest APIs, and microservices. She was part of the middleware team where she also gained proficiency in backend technologies. She has contributed to automations at her previous work where she picked up skills in scripting languages. Her work also entailed DevOps which helped her knowledge of CI/CD technologies such as Kubernetes, Jenkins, BitBucket, git, and Gerrit. She has taken courses in Computer Architecture, Advanced Operating Systems, Design and Analysis of Algorithms, Advanced Computer Networks, Performance of Computer Networks and Systems, Cyber-physical Systems and Critical Infrastructure, Microprocessors and Embedded Systems. She has a programming and scripting background in Java, C, C#, python, shell, and multiple frameworks.

Haoyan Li is in her second semester of the Master's Degree of Computer Science in University of Texas in Dallas. She has work experience in building, testing and maintenance of the Manufacturing Execution System in industry. She was also part of the team of training defecting image detection and classification, where she gained experience in programming and scripting in Python, Tensorflow, Keras, scikit-learn, as well as implementing APIs. She is currently taking Data and Application Security course and is expecting to learning more about access control, security policies and models in the cybersecurity field. She is also experienced in Java and C++.

References

- [1] Intezer. (2021, December 22). *The role of malware analysis in cybersecurity*. Intezer. <https://www.intezer.com/blog/malware-analysis/the-role-of-malware-analysis-in-cybersecurity/>
- [2] Cassiusm, P., Annelie, H., Olivier, Z., and Lamine, N. (2021, August 17-20). Accurate and Robust Malware Analysis through Similarity of External Calls Dependency Graphs (ECDG). IWCC'21, All-Digital Conference
- [3] Konrad, R., Philipp, T., Carsten, W., and Thorsten, H. (2011). Automatic Analysis of Malware Behavior using Machine Learning. *Journal of Computer Security*. <http://www.iospress.nl>,